

# **Práctica 1 - Inteligencia Artificial**

**Luca Bellu e Iván Domínguez**

# Roles según CRediT

| Rol CRediT                                             | Ejercicio 1 (WalkSAT) | Ejercicio 2 (Z3) |
|--------------------------------------------------------|-----------------------|------------------|
| Conceptualización (elección y motivación del problema) | L                     | L                |
| Metodología (formalización lógica)                     | I                     | I                |
| Software (implementación)                              | IA                    | IA               |
| Validación (verificación de soluciones, pruebas)       | L                     | L                |
| Visualización (tableros y figuras)                     | L                     | L                |
| Redacción: borrador original                           | I                     | I                |
| Redacción: revisión y edición                          | I                     | I                |

# Problema 1

# Problema de las $N$ reinas.

- Evalúa un posible universo en el que todas las casillas en un tablero de ajedrez  $N \times N$  están siendo atacadas por una reina.
- Sabiendo que es un problema con soluciones para cualquier  $N$ , emplearemos un SAT-Caminar por cuestiones de eficiencia.
- Referencia: Rostami, T., & Bright, C. (2026). Queen domination by SAT solving. arXiv. <https://arxiv.org/abs/2508.11945>.

# Planteamiento del problema

## Restricciones

Llamaremos  $x_{i,j}$  a la variable proposicional que es verdadera cuando hay una reina en  $i, j$  y negativa en caso contrario.

- Al menos una reina por fila  $i$ , como máximo una reina por fila  $i$ .

$$x_{i,j1} \vee x_{i,j2}, \vee \cdots \vee x_{i,jN} \equiv \bigvee_{j=1}^N x_{i,j} , \neg(x_{i,j1} \wedge x_{i,j2} \wedge \cdots \wedge x_{i,jN}) \equiv \bigvee_{j=1}^N \neg x_{i,j}$$

- Al menos una reina por columna  $j$ , como máximo una reina por columna  $j$ .

$$x_{i1,j} \vee x_{i2,j}, \vee \cdots \vee x_{iN,j} \equiv \bigvee_{i=1}^N x_{i,j} , \neg(x_{i1,j} \wedge x_{i2,j} \wedge \cdots \wedge x_{iN,j}) \equiv \bigvee_{i=1}^N \neg x_{i,j}$$

# Planteamiento del problema

## Más restricciones

- Como máximo una reina por diagonal principal. Todas las celdas comparten la misma diferencia de coordenadas  $i_1 - j_1 = i_2 - j_2 = \dots = i_N - j_N$ .
- Como máximo una reina por diagonal secundaria. Todas las celdas comparten misma suma de coordenadas  $i_1 + j_1 = i_2 + j_2 = \dots = i_N + j_N$ .
- La restricción única será entonces  $|i_k - i_{k+1}| = |j_k - j_{k+1}|$  y entonces

$$\bigvee_{k=1}^N \neg x_{ik,jk}$$

# SAT Solver: SAT-Caminar

Imaginemos la  $BC = \{C_1 \equiv A \vee B, C_2 \equiv \neg A \vee C, C_3 \equiv \neg B \vee \neg C\}$

- Paso 1: asignación inicial aleatoria. Por ejemplo  $A = F, B = F, C = F$
- Paso 2: evaluación del modelo.  $C_1$  no se cumpliría.
- Paso 3: selección e intercambio. Con probabilidad  $p$  arbitraria, elige entre:
  - Estrategia de pasada-aleatoria: invierte  $A$  o  $B$  aleatoriamente.
  - Estrategia de min-conflictos: evalúa qué cambio funcionará sin romper la satisfacibilidad de otras cláusulas. Cambiar  $A$  rompe otras,  $B$  no.

Se han usado las restricciones ya  
optimizadas del paper.

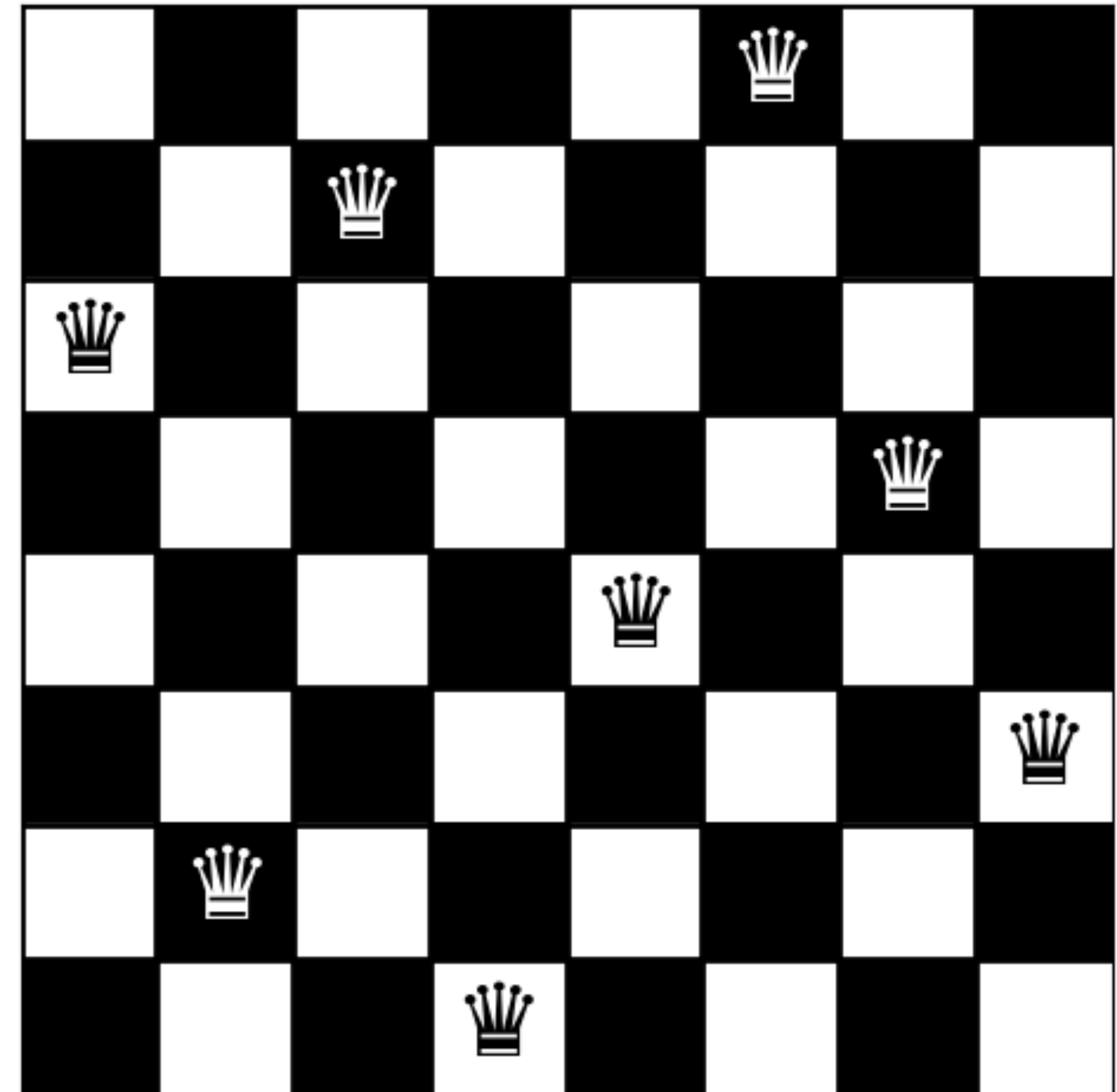
# Resultados (1)

N=8

Tablero convencional

Todas las casillas están atacadas

Solución SAT-CAMINAR para un tablero 8x8



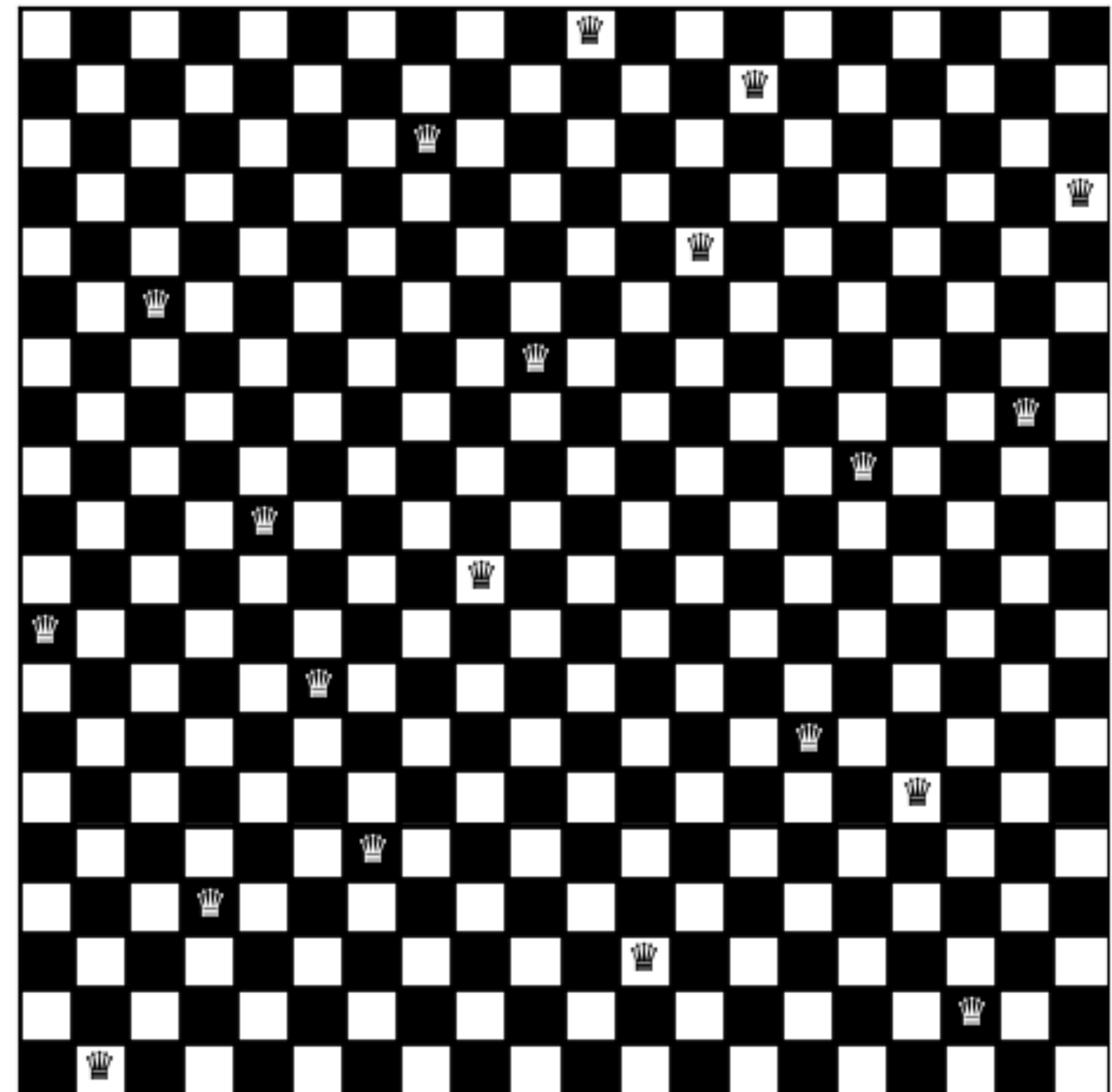
# Resultados (2)

N=20

Unos 3 segundos.

Todas las casillas están siendo  
atacadas.

Solución SAT-CAMINAR para un tablero 20x20



# **(Casi) siempre satisfacible**

- Está demostrado que existe una solución para cualquier  $N = 1, N \geq 4$ .
- Por eso podemos usar el SAT-Caminar.
- SAT-Caminar es incompleto, que no dé una solución no quiere decir que no exista.

# Ejercicio 2

# Mismo problema, más ambicioso

- Usando lógica de primer orden podemos atacar un problema parecido pero más ambicioso.
- Eliminamos la restricción de no amenazarse entre sí.
- Añadimos la de optimización (encontrar el menor número de reinas).
- Más nativo en lógica de predicados:
  - Miles de cláusulas independientes  $\rightarrow$  axiomas universales.

# Razones

- Escalabilidad: la FNC es menos nativa y depende del tamaño del tablero, el número de variables es  $O(N^2)$  y el número de cláusulas  $O(N^3)$ .
- Con los cuantificadores  $\forall$  y  $\exists$  se codifican reglas universales. Es fácil ver que  $\lim_{k \rightarrow \infty} \bigvee_{n=i}^k \equiv \exists$  y  $\lim_{k \rightarrow \infty} \bigwedge_{n=i}^k \equiv \forall$ .
- Ya no hay que pre-calcular las posiciones de las diagonales.

# Formalización

- Predicados:  $\text{Reina}(x, y)$     $\text{Ataque}(x_1, y_1, x_2, y_2)$
- Definición geométrica del ataque:
  - Compartición de filas, columnas o diagonales es definido ataque.
  - $\forall x_1 \forall y_1 \forall x_2 \forall y_2 (\text{Ataque}(x_1, y_1, x_2, y_2) \iff (x_1 = x_2 \vee y_1 = y_2 \vee |x_1 - x_2| = |y_1 - y_2|))$
- Restricción de dominación:
  - Toda casilla está siendo atacada por alguna reina.
  - $\forall x \forall y \exists i \exists j (\text{Reina}(i, j) \wedge \text{Ataque}(i, j, x, y))$
- Restricción de cardinalidad:
  - Para el problema del tablero  $5 \times 5$ , hay 3 “asientos” disponibles:  $(r_1, c_1), (r_2, c_2), (r_3, c_3)$
  - $\exists r_1 \exists c_1 \exists r_2 \exists c_2 \exists r_3 \exists c_3 \forall x \forall y (\text{Reina}(x, y) \implies ((x = r_1 \wedge y = c_1) \vee (x = r_2 \wedge y = c_2) \vee (x = r_3 \wedge y = c_3)))$

# Desventajas

- Tiempo:
  - Aunque lógicamente sea más fácil de entender, la lógica de primer orden tiene mayores exigencias computacionales.
- Satisfabilidad:
  - El problema trata de buscar un mínimo, no solo de comprobar la satisfabilidad de la solución.
  - No encuentra “una” solución, encuentra “la” solución.

# Microsofts *python z3-solver*

- Framework de python que implementa un motor SMT (Satisfiability Modulo Theories).
- Integra teorías matemáticas de fondo; no resuelve por fuerza bruta.
- Eso le permite integrar de forma nativa los cuantificadores  $\forall$  y  $\exists$ .
- Tambien reconoce estructuras geométricas, por lo que las diagonales no son pre-calculadas, son directas.

$\gamma \equiv$  número de reinas

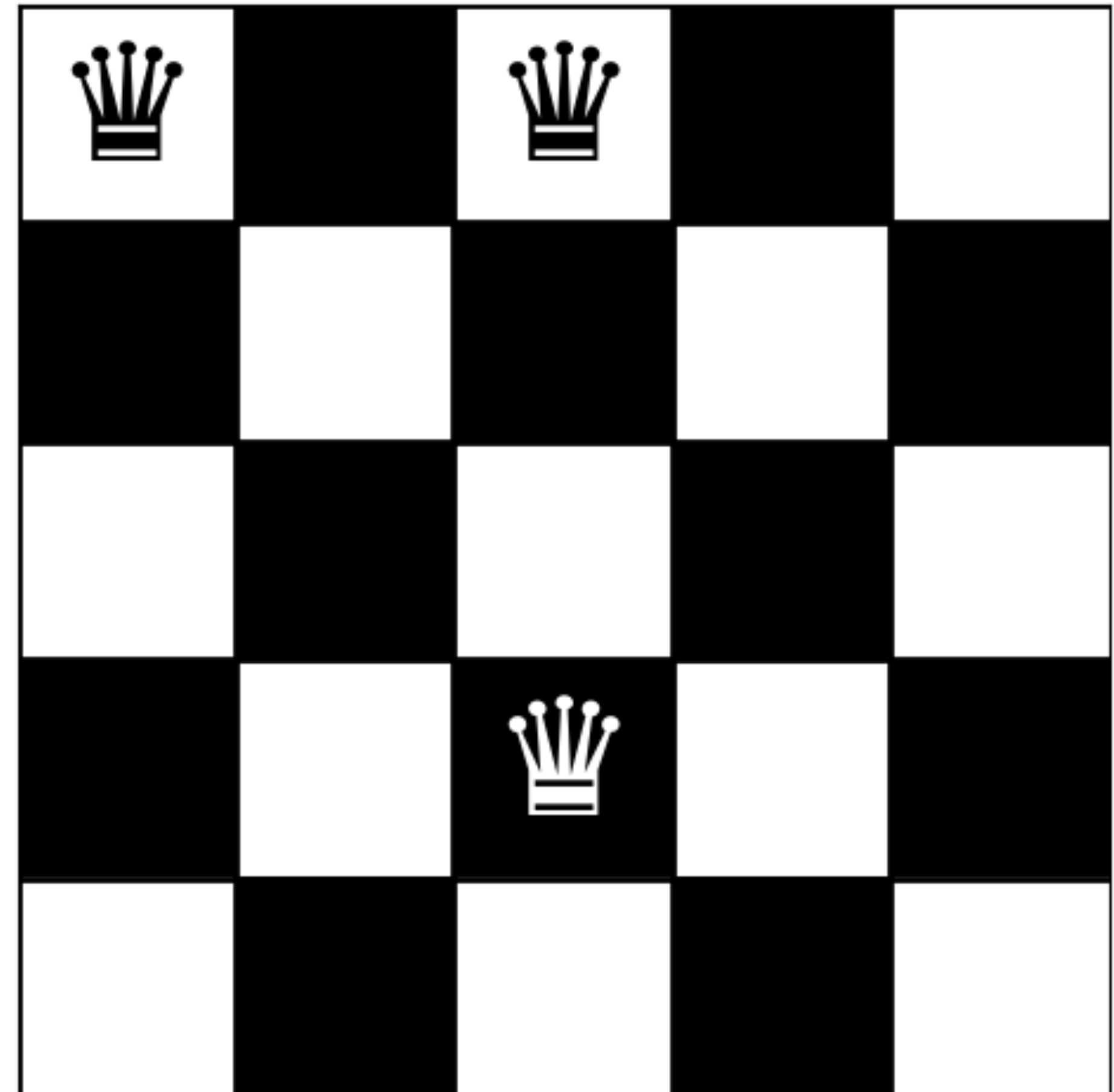
# Resultados

Todas las casillas son atacadas.

Las reinas se atacan entre sí.

¿Qué pasaría con  $\gamma \neq 3$ ?

Solución LPO (Z3): 3 reinas (tablero 5x5)



$$\gamma = 2$$

- INFO: Tablero 5x5: probando con a lo sumo 1 reina(s)...
- INFO: gamma=1: INSATISFACIBLE (0.01 s). No hay forma de dominar el tablero.
- INFO: Tablero 5x5: probando con a lo sumo 2 reina(s)...
- INFO: gamma=2: INSATISFACIBLE (0.16 s). No hay forma de dominar el tablero.
- WARNING: Ningún gamma  $\leq 2$  domina el tablero

$$\gamma = 4$$

- INFO: Tablero 5x5: probando con a lo sumo 1 reina(s)...
- INFO: gamma=1: INSATISFACIBLE (0.01 s). No hay forma de dominar el tablero.
- INFO: Tablero 5x5: probando con a lo sumo 2 reina(s)...
- INFO: gamma=2: INSATISFACIBLE (0.15 s). No hay forma de dominar el tablero.
- INFO: Tablero 5x5: probando con a lo sumo 3 reina(s)...
- INFO: gamma=3: SATISFACIBLE (1.26 s). Reinas en [(0, 0), (0, 2), (3, 2)]; verificación independiente: CORRECTA.
- Nunca llega a 4 porque con 3 reinas ya hay una solución satisfacible.

# No siempre satisfacible

- Por eso se prueba en el espacio  $\gamma = \{1, 2, \dots, N\}$ ,  $N < \infty$  (también sabemos que hay solución, pero no sabemos en qué valor de  $N$ , que además tiene que ser mínimo).
- Con valores menores que  $N$ , el problema no es satisfacible, como vimos en el ejemplo  $\gamma = 2$ .
- Y una vez encuentra  $N$ , nunca prueba en valores mayores, pues se requiere que  $N$  sea mínimo.

# Conclusiones (1)

- Completo vs. incompleto:
  - SAT-Caminar escala bien pero solo sirve si sabes que hay solución.
  - $Z_3$  es más lento pero puede demostrar que  $k=1$  y  $k=2$  son insatisfacibles, y eso es lo que prueba que 3 es el mínimo para  $N = 5$ .
- Expresividad vs. escalabilidad:
  - LPR: es más fácil de entender pero no escala bien para  $N$  grande.
  - LPO: es más compleja, pero a cambio no depende de  $N$  con un  $\gamma$  bien escogido.
- La elección del formalismo depende del problema, no hay uno mejor.

# Conclusiones (2)

|             | Problema 1                               | Problema 2                              |
|-------------|------------------------------------------|-----------------------------------------|
| Objetivo    | Colocar N reinas sin atacarse            | Mínimo de reinas que dominen el tablero |
| Tipo        | Satisfacción                             | Optimización                            |
| Formalismo  | CNF (proposicional)                      | Lógica de primer orden                  |
| Herramienta | SAT-Caminar (búsqueda local, incompleta) | Z3 (SMT, completo)                      |
| Resultado   | N=8 y N=20 (~3 s)                        | $\gamma(5)=3$ (1,26 s)                  |

# Limitaciones

- Problema 2 solo probado en  $5 \times 5$ ; el número de "asientos" está fijado a mano.
- Tiempos medidos sobre problemas distintos, no comparables directamente.
- SAT-Caminar es aleatorio: conviene reportar tiempo medio en varias ejecuciones.